Smart Drug Recognition





Presented by: **GROUP 8**

Dataset Preparation & Data Pipeline

Presented by: **YOUSSEF ABDALLA**



Project Overview & Problem Definition

- The main problem we are addressing is the detection of **counterfeit medicine packages**, which is a serious global health issue.
- Fake medicines can look extremely similar to original ones, making manual inspection unreliable. Therefore, our goal is to build an automated system that can classify drug package images as **original or counterfeit** using deep learning.

Real



Fake



Dataset Description

- For this project, we used a real-world dataset collected for counterfeit drug detection.
- The dataset consists of approximately **661 images**, including **421 original** and **240 counterfeit** medicine packages.
- The images include boxes, strips, labels, and different package types, which makes the dataset realistic and challenging.



https://www.kaggle.com/datasets/surajkumarjha1/fake-vs-real-medicine-datasets-images

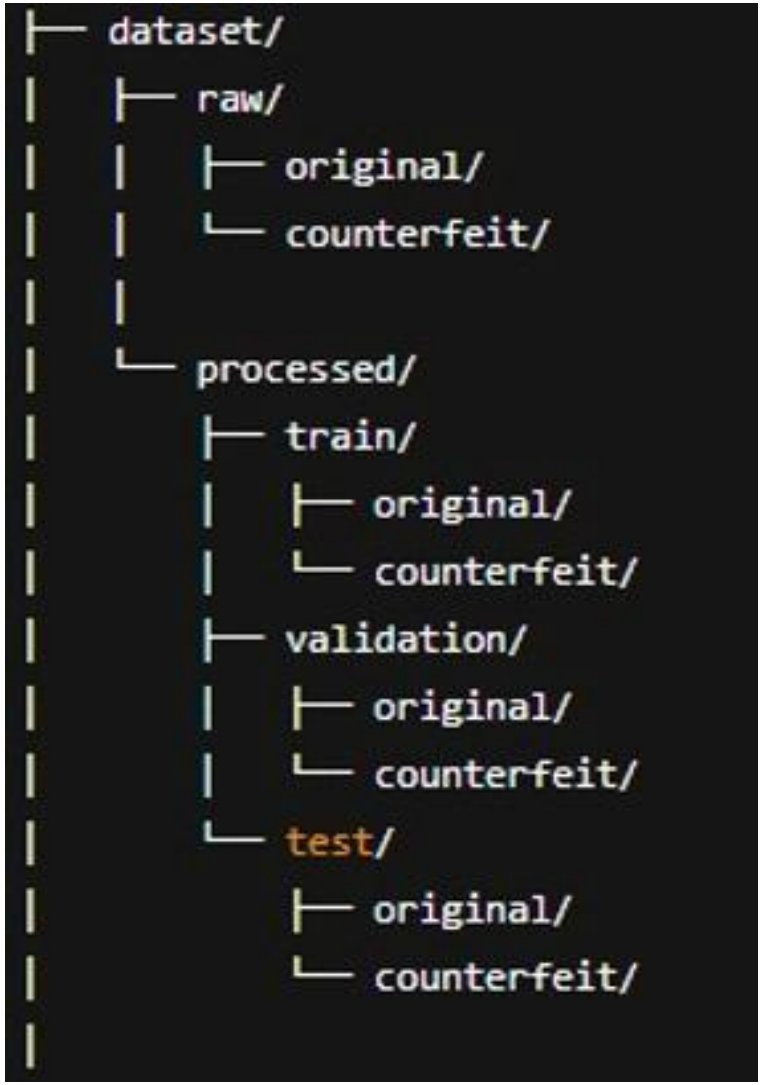
Data Challenges

- One of the main challenges we faced was that the dataset was **not very large**, and the visual differences between original and counterfeit packages are often very subtle.

- Another critical challenge was avoiding **data leakage**, where the same image could accidentally appear in both training and testing sets, which would give misleadingly high accuracy.



Data Pipeline & Splitting Strategy



- To solve these issues, we designed a **custom data pipeline**.
- We split the dataset using a **70% training, 15% validation, and 15% testing** strategy.
- Inside each split, images are further divided into original and counterfeit folders.
- Most importantly, **no image appears in more than one split**, which guarantees fair evaluation.

Dataset Preparation Script

We implemented an automated script called **prepare_dataset.py**

This script performs five key operations

Merge
Shuffle
Split
Rename
Summary generation.

This ensured a clean, reproducible, and portable dataset preparation process that works on any machine.



Model Selection & Training Strategy

Presented by: **AHMED ALI**

220303975



Why Model Selection Matters

Once we had our dataset ready, we faced a critical decision: How do we build a model that is smart enough to detect very subtle differences between real and fake drugs?

We initially considered building a Convolutional Neural Network (CNN) from scratch. However, we quickly realized that training a CNN from scratch requires tens of thousands of images to work effectively. Since our dataset consists of only a few hundred images, building a model from scratch would likely lead to **overfitting** or **underfitting** — meaning the model wouldn't generalize well to new data.

To solve this, we chose **Transfer Learning**.



What is Transfer Learning?

Transfer Learning is a technique where we use a model that has already been trained on a massive dataset, like ImageNet. This pre-trained model already knows how to recognize shapes, edges, and colors.

Instead of teaching the computer how to 'see' from the beginning, we borrow this knowledge. This helps us in three ways:

1. It reduces our training time.
2. It improves performance, even with our small dataset.
3. It prevents the model from overfitting (memorizing the data).



Why MobileNetV2 Specifically

After choosing Transfer Learning, we selected **MobileNetV2** as our specific model. Why did we choose MobileNetV2? We chose it because it is **lightweight and efficient**.
Our project goal is to have a system that can work in the real world, like on a mobile app or a web interface. We did not want a huge, slow model. MobileNetV2 gives us the best balance: it is fast enough for real-time prediction, but it is also strong enough to extract the complex features we need to spot fake drugs.

```python
def build_model(img_size=(224, 224)):
    # Base model: MobileNetV2 pretrained on ImageNet (Transfer Learning)
    base = tf.keras.applications.MobileNetV2(
        input_shape=(img_size[0], img_size[1], 3),
        include_top=False,
        weights="imagenet",
    )
    base.trainable = False  # freeze feature extractor for first training phase

    inputs = tf.keras.Input(shape=(img_size[0], img_size[1], 3))
    # MobileNetV2 expects inputs in [-1, 1], so we use its preprocess function
    x = tf.keras.applications.mobilenet_v2.preprocess_input(inputs)
    x = base(x, training=False)
    x = tf.keras.layers.GlobalAveragePooling2D()(x)
    x = tf.keras.layers.Dropout(0.2)(x)
    outputs = tf.keras.layers.Dense(1, activation="sigmoid")(x)  # binary classification
    model = tf.keras.Model(inputs, outputs)
    return model
```

Training Strategy

Phase 1 is Feature Extraction. In this phase, we **froze** the base MobileNetV2 model. This means we locked the pre-trained weights so they wouldn't change. We only trained the new classification layers that we added on top.

Why did we do this? We wanted the model to use its existing visual knowledge—like recognizing edges, shapes, and textures — without destroying that information while it learned the basic difference between our 'original' and 'counterfeit' classes. This stabilized the early training process."



Fine-Tuning Strategy

Phase 2 is Fine-Tuning. Once the model stabilized, we moved to the second phase. Here, we **unfroze the last 30 layers** of the MobileNetV2 model.

This is the most critical part. We used a very small learning rate (specifically **0.00001**) to carefully adapt the model's high-level features.

By unfreezing these specific layers, we allowed the model to learn the very specific, subtle details of our domain—such as slight differences in **font styles, color consistency, and print quality** on the medicine boxes. This fine-tuning is what allowed us to achieve high accuracy even with visually similar packages.



Training Analysis, Evaluation & Performance

Presented by: **ALI AL-MUSTAFA**



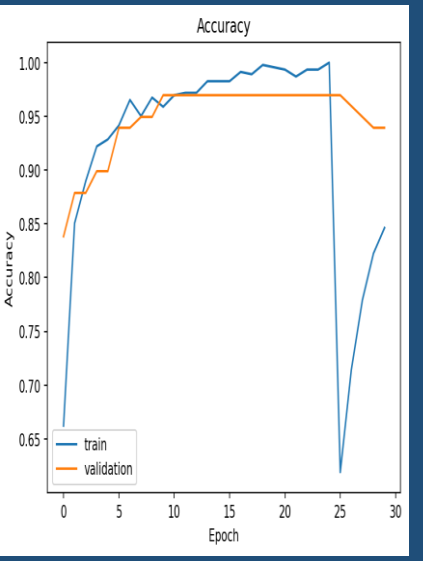
Training Monitoring

During training, we closely monitored training and validation accuracy and loss.

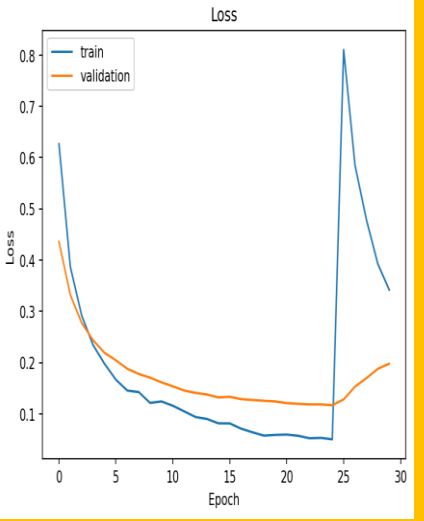
These curves help us understand whether the model is learning correctly or suffering from overfitting or underfitting.

In our case, both training and validation curves showed a consistent improvement, indicating stable learning.

ACUURACY



LOSS



Fine-Tuning Behavior

- At the beginning of the fine-tuning phase, a temporary drop in training accuracy and a spike in loss were observed.
- This behavior is expected when previously frozen convolutional layers are unfrozen.
- After several epochs, the model stabilized and continued improving.
- This confirms that the pretrained features successfully adapted to the new domain.



Test Set Evaluation

To evaluate real-world performance, we tested the model on an unseen test set.

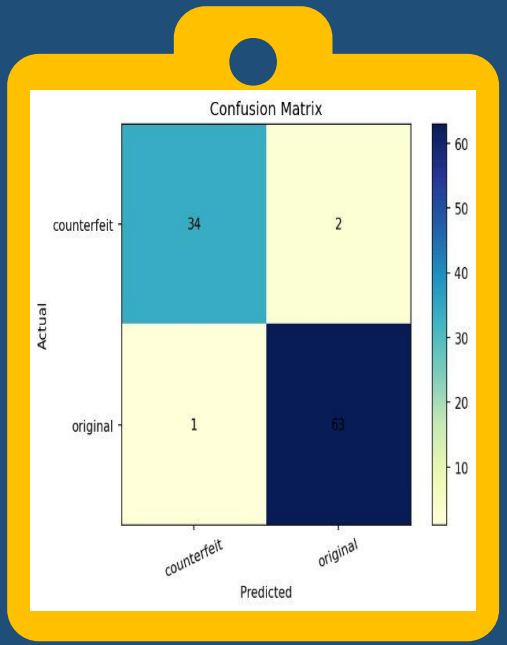
The model achieved an accuracy of **97%**, demonstrating strong generalization beyond the training data.



Confusion Matrix Analysis

- The confusion matrix provides deeper insight than accuracy alone.
- It shows how many original and counterfeit samples were correctly and incorrectly classified.
- Importantly, the number of false positives—where counterfeit drugs are classified as original—was very low.
- In medical applications, minimizing false positives is more important than maximizing accuracy.

	precision	recall	f1-score	support
counterfeit	0.9714	0.9444	0.9577	36
original	0.9692	0.9844	0.9767	64
accuracy			0.9700	100
macro avg	0.9703	0.9644	0.9672	100
weighted avg	0.9700	0.9700	0.9699	100



Classification Metrics

Precision, recall, and F1-score were all around **97%** for both classes.

This indicates balanced performance and confirms that the model does not favor one class over the other.



Deployment, Model Inference, Thresholding & Conclusion

Presented by: **AHMED BAHGAT**



From Training to Inference

After training and evaluating the deep learning model, the next step is **model inference**, which means using the trained network to make predictions on new, unseen images.

In this phase, the model no longer learns. Instead, it applies the learned weights to classify incoming images as original or counterfeit.

This step represents the real-world usage of deep learning models, where accuracy, speed, and reliability become critical.



Decision Threshold & Risk Awareness

The model outputs a probability value between 0 and 1 using a sigmoid activation function.

To convert this probability into a final decision, a decision threshold is applied.

While 0.5 is a common default threshold, in medical applications it is sometimes adjusted to reduce the risk of dangerous misclassification.

In our case, misclassifying a counterfeit drug as original is more dangerous than the opposite, so threshold selection becomes an important deep learning design decision.

Threshold tuning allows prioritizing safety over raw accuracy in sensitive applications.



Confidence Score & Model Interpretability

Instead of returning only a class label, the system also provides a confidence score derived from the model's output probability.

This confidence score increases model interpretability and helps users understand how certain the model is about its decision.

In critical domains like healthcare, interpretability is as important as accuracy, because users must be able to trust and assess AI decisions.



Model Deployment & Integration

The trained deep learning model was saved and later loaded for inference without retraining.

This separation between training and deployment ensures reproducibility and portability across different systems.

The interface acts as a wrapper around the deep learning model, Enabling real-time predictions while keeping the model logic unchanged.





LIVE DEMO

THANKS FOR YOUR ATTENTION

